

Security Assessment

Workspace Program

August 26, 2026

Overview

Project Summary

Project Name	Workspace Program
Language	Rust (Anchor Framework)

Audit Summary

Delivery Date	August 26, 2026 UTC
Audit Methodology	Static Analysis, Manual Review

Vulnerability Summary

Vulnerability Level	Total
Critical	0
Major	0
Medium	1
Minor	1
Informational	2

Table of Contents

Summary

Overview

Project Summary

Audit Summary

Vulnerability Summary

Audit Scope (Scope includes all provided Rust files)

Findings

Finding-01 : Denial of Service (DoS) via Forced Vault Deposits

Finding-02 : Permanent Account Lock Due to Dust Accumulation

Finding-03 : Sub-optimal Seed Dependency in PDA Validation

Finding-04 : Hardcoded Protocol Wallet Restricts Flexibility

Findings

4 Total Issues

Medium: 1, Minor: 1, Informational: 2

Title	Category	Severity
Denial of Service (DoS) via Forced Vault Deposits	Logical Flaw / Griefing	Medium
Permanent Account Lock Due to Dust Accumulation	Math / Arithmetic	Minor
Sub-optimal Seed Dependency in PDA Validation	Anchor Best Practices	Informational
Hardcoded Protocol Wallet Restricts Flexibility	Design Pattern	Informational

Denial of Service (DoS) via Forced Vault Deposits

Category	Logical Flaw / Griefing
Severity	Medium

Description

The `close_project` instruction requires that the `vault` PDA balance is less than or equal to the rent minimum plus a strict dust tolerance. This rule is applied globally, regardless of whether the project is `Open` or `Finalized`.

Because the `vault` is a native SOL System Account, any user can transfer lamports to it directly outside of the program's normal operations. A malicious actor can intentionally send a small amount of SOL (exceeding the dust tolerance) to the vault of an `Open` project. Since the project is open, the funds cannot be distributed, and because the vault is no longer "empty", the creator is permanently blocked from closing the project and recovering their account rent. This results in a Denial of Service and permanent lock of the Project PDA.

Recommendation

Adjust the closure logic to handle Open and Finalized states differently. For Open projects, allow the creator to close the project regardless of the vault balance, as they cannot finalize it unilaterally anyway. During the closure of an Open project, any unexpected lamports deposited into the vault can either be swept to the creator or routed to a protocol treasury to disincentivize griefing attacks. Keep the strict dust tolerance check only for Finalized projects.

Permanent Account Lock Due to Dust Accumulation

Category	Math / Arithmetic
Severity	Minor

Description

During the `distribute` execution, member payout amounts are calculated using integer division against a 10,000 basis points scale. Due to truncation, each distribution call can leave a small remainder of lamports (dust) in the vault (up to roughly the maximum number of members in lamports).

While the protocol implements a `DUST_TOLERANCE_LAMPORTS` check (1,000 lamports) to allow closing projects with dust, this tolerance is static. If a project undergoes recurring distributions over time (e.g., more than 150 distributions), the accumulated truncation dust can exceed the 1,000-lamport limit. Once this threshold is surpassed, the `close_project` instruction will persistently fail, permanently locking the rent exemption of the finalized project.

Recommendation

Revise the distribution math to ensure zero dust is left behind for member payouts. Track the running total of lamports calculated for members in the loop. For the final member in the array, instead of recalculating their share with division, assign them the remaining balance of `after_fee` minus the already calculated payouts of previous members. This guarantees all funds are correctly exhausted down to the exact lamport.

Sub-optimal Seed Dependency in PDA Validation

Category	Anchor Best Practices
Severity	Informational

Description

In the `Fund` and `Distribute` context structs, the `Project` account defines its expected seeds using its own internal fields (e.g., `project.creator.as_ref()`).

Anchor handles this by deserializing the supplied account data first, and then deriving the seeds to verify the PDA address. While technically functional because Anchor will enforce that the address matches the data it holds, relying on an account's own internal data to derive its verification seeds is an unconventional pattern. It theoretically allows a user to pass a valid but entirely synthetic program-owned account. Due to the architecture, this does not pose a direct financial risk here (as the vault PDA derivation relies on the project's public key), but it deviates from standard Solana deterministic account resolution practices.

Recommendation

Refactor the account validation structs to remove self-referential seed dependencies. You can achieve this by passing the required variables (like the `project_id`) as instruction arguments, and validating the `creator` by passing it as a separate account in the context, or by utilizing the `has_one = creator` constraint to enforce logical relationships explicitly.

Hardcoded Protocol Wallet Restricts Flexibility

Category	Design Pattern
Severity	Informational

Description

The program relies on a statically defined constant, `PROTOCOL_WALLET`, to define where the 2% protocol fee is routed. During `create_project`, the user must provide an exact match for this pubkey, which is then stored in the project state.

If the protocol's treasury or operational wallet ever needs to be rotated (e.g., due to a security key compromise, team restructuring, or migration to a multisig), the entire Solana program will need to be recompiled and upgraded. This introduces unnecessary friction for long-term operational maintenance.

Recommendation

Consider implementing a global configuration state account (PDA) initialized once by the program upgrade authority. This config account should store the active protocol wallet address. Read this state during initialization and distribution instructions. This will permit administrative rotation of the fee-collection wallet without requiring a program code upgrade.

Summary

This report summarizes the security assessment of the Workspace Program written in Rust using the Anchor framework. The assessment involved a comprehensive examination of the provided source code using Static Analysis and Manual Review techniques.

The audit focused on identifying potential vulnerabilities, logical flaws, and adherence to Solana and Anchor best practices. Key areas of focus included:

- Account validation and access control.
- Handling of token transfers and CPI calls.
- Arithmetic operations to prevent overflows/underflows.
- State management, lifecycle events, and data integrity.

The assessment identified several issues categorized by severity. A **Medium** severity issue was found regarding an external grieving vector where unauthorized deposits to an open vault can prevent project closure, locking rent funds. A **Minor** issue regarding precision loss resulting in continuous dust accumulation was also identified, which could unexpectedly break project closure for highly active projects. Two **Informational** findings highlight areas for potential improvement regarding seed validation practices and overall contract upgradability.

Overall, the core business logic, share validation, and member authorization checks are well-structured and secure. The contract actively utilizes safe arithmetic boundaries. By addressing the denial-of-service edge cases and tightening the distribution mathematics, the program's resilience and user experience will be greatly enhanced.

Disclaimer

For the purposes of this disclaimer:

- "*Noah AI*" refers to the product currently operated by its owners and developers, including but not limited to its creators, contributors, affiliates, officers, employees, contractors, and agents.
 - "*Plena*" refers to Plena Finance Digital LLC, including but not limited to its affiliates, officers, employees, contractors, and agents.
 - "*User*" refers to the recipient of this report, whether an individual or an entity, and any of its affiliates, officers, employees, contractors, or agents who access, review, or act upon the contents of this report.
-

AI-Generated Content

This report is *entirely generated by artificial intelligence (AI)* without human authorship, review, or verification by Noah AI or Plena. AI-generated content is inherently probabilistic and may contain inaccuracies, false positives, or false negatives. *The User is solely and fully responsible* for reviewing, verifying, and validating all information before relying on it or taking any action.

No Responsibility or Liability

To the maximum extent permitted by law, *Noah AI and Plena, and their respective owners, operators, affiliates, officers, employees, contractors, and agents*, **accept no responsibility or liability** for:

- The accuracy, completeness, legality, reliability, or security of this report
 - Any hacks, exploits, unauthorized access, security breaches, or vulnerabilities
 - Any loss of funds, data, or digital assets
 - Any direct, indirect, incidental, consequential, punitive, or special damages of any kind, *whether in contract, tort, or otherwise*, even if advised of the possibility of such damages
-

No Endorsement or Advice

Nothing in this report constitutes or should be interpreted as an endorsement, recommendation, disapproval, investment advice, financial advice, legal advice, or guarantee of performance.

No Guarantee of Regulatory Compliance

Nothing in this report constitutes a guarantee by Noah AI or Plena of regulatory compliance, legal validity, or adherence to applicable laws in any jurisdiction.

No Contract Formation

Use of this report does not create any contractual obligations between the User and Noah AI, Plena, or their respective owners, beyond those explicitly stated in a written agreement between the parties.

Third-Party Dependencies

References to third-party technologies, services, or platforms are for informational purposes only and do not constitute endorsements or assurances by Noah AI or Plena of their reliability, security, or legality.

Risk Disclaimer

Blockchain technology, AI systems, and cryptographic assets are highly experimental and carry substantial risks. The User remains solely responsible for conducting independent due diligence, obtaining professional advice, and ensuring compliance with all applicable laws and regulations before acting on any information in this report.